



C#

○ Topics # 0 of 0 Topic(s) Planned.

Synthèse complète des points du document sur l'introduction à WPF en C# :

1. Objectifs de la formation

- Créer des **applications Windows** avec une interface graphique.
 - Aborder l'**ergonomie** et le **design** pour des interfaces intuitives.
 - Concevoir l'**architecture** d'une application en séparant l'interface et la logique.
 - Maîtriser l'utilisation des **composants graphiques** (boutons, labels, etc.).
 - Implémenter la **gestion des événements** (clics, déplacements de souris...).
-

2. Historique des technologies Windows

- **Windows Forms (2002)** : Première interface graphique pour .NET, toujours compatible avec Windows 10.
- **WPF (2006)** : Windows Presentation Foundation, basé sur .NET 3.0 :
 - Séparation **UI** (XAML) et **logique métier** (C#).
 - Support **2D/3D**, animations, multimédia via l'API **DirectX**.
 - Meilleure gestion du **databinding** (liaison des données).
- **UWP (2012)** : Universal Windows Platform :
 - Support pour les **tablettes** (écran tactile) et **Windows Store**.

- Applications universelles sur **Windows 10** (Desktop + mobile).
 - **Xamarin Forms** : Développement **cross-platform** (Android, iOS, UWP).
-

3. Ergonomie et design d'une application

- Interface typique :
 - Une **fenêtre principale** (GUI) pour les tâches principales.
 - Des **fenêtres secondaires** pour des tâches spécifiques (ex : boîtes de dialogue).
 - Éléments standards : Titre, boutons (fermer, minimiser, maximiser), menu, zones de texte.
 - Bonnes pratiques :
 - Créer des **interfaces intuitives** en respectant les conventions d'usage.
 - Garder une **cohérence** entre les différentes sections de l'application.
-

4. Structure d'une application WPF

- **XAML (eXtensible Application Markup Language)** :
 - Langage de description de l'UI, proche de XML/HTML.
 - Exemple :

```
<Button Background="Blue" Content="Click Me"/>
```

- **App.xaml** : Définition de l'application.
 - **MainWindow.xaml** : Interface de la fenêtre principale.
 - **.xaml.cs** : Code "behind" en C# (logique métier).
-

5. Première application WPF (Hello World)

1. Créer un **projet WPF** dans Visual Studio.
2. Structure de base :
 - **App.xaml** : Définit la classe principale.
 - **MainWindow.xaml** : Décrit l'interface principale.

3. Exemple de **Hello World** :

- Ajout d'un `Label` rouge via XAML.
 - Utilisation de la **Toolbox** pour le glisser-déposer des composants.
-

6. Gestion des événements

- Un **événement** est déclenché par une action (clic, déplacement de la souris...).
- Exemple : Bouton qui change le texte :

```
private void ClickMeButton_Click(object sender, RoutedEventArgs e)
{
    label.Content = "Bonjour le monde !";
}
```

- **Association** d'un événement à un bouton dans XAML :

```
<Button Click="ClickMeButton_Click" Content="Click Me"/>
```

7. Bonnes pratiques de codage

- Nommer les fichiers et les **contrôles** de manière explicite.
 - Exemple :
 - `MainWindow.xaml` → `HelloWorldWindow.xaml`
 - `Class1.cs` → `Personne.cs`
-

8. Architecture de l'application WPF

- **Assemblies et namespaces** :
 - `System.Windows` : Racine de WPF.
 - `System.Windows.Controls` : Boutons, menus, etc.
 - `System.Windows.Data` : Liaison de données (databelinding).
- **Héritage principal** :
 - `Application` → Classe principale.

- **Window** → Fenêtre principale.

9. Gestion des contrôles WPF

- Les **contrôles** sont les éléments graphiques de l'interface.
- Exemple de création d'un bouton :
 - En C# :

```
Button btn = new Button();  
btn.Content = "Click Me";
```

- En XAML :

```
<Button Content="Click Me" Width="100" Height="40"/>
```

10. Dialog Boxes (Boîtes de dialogue)

- **OpenFileDialog** : Sélection de fichiers à ouvrir.

```
Microsoft.Win32.OpenFileDialog dlg = new OpenFileDialog();  
dlg.Filter = "Text documents (.txt)|*.txt";  
dlg.ShowDialog();
```

- **MessageBox** : Affichage de messages.

```
MessageBox.Show("Erreur", "Message", MessageBoxButton.OK, Message  
BoxImage.Error);
```

11. Gestion du Layout (Disposition des composants)

1. **Canvas** : Positionnement **absolu** (X, Y).
2. **WrapPanel** : Réorganise les composants **dynamiquement**.
3. **StackPanel** : Empile les éléments en **colonne** ou en **ligne**.
4. **Grid** : Organisation en **grille**, similaire aux **tables HTML**.
5. **DockPanel** : Attache les composants aux **bords**.

6. **ScrollView** : Ajoute des **barres de défilement**.

12. Exemple d'application : Gestion de personnes

- Interface :
 - TextBox pour le nom et prénom.
 - Bouton pour valider les données.
- Vérification des champs :

```
if (NomTB.Text == "" || PrenomTB.Text == "")
{
    MessageBox.Show("Erreur");
}
```

- Personnalisation de l'affichage :
 - Changer la **couleur** avec `Brushes.Red`.
 - Modifier le **contenu** du bouton dynamiquement.
-

13. Conseils pour structurer un projet WPF

- **Séparer** les différentes fonctionnalités :
 - Projets de type "**Librairies de classes**" pour la logique métier.
 - Projets "**Application**" pour l'interface graphique.
 - Exemple :
 - `PeopleLib` : Gestion des personnes.
 - `HelloWorld` : Application principale.
-

Ce résumé couvre l'ensemble des concepts abordés : de l'introduction au développement complet d'une application WPF en C#. Si tu souhaites plus de détails sur un point spécifique, fais-le-moi savoir ! 🚀

Structure recommandée des dossiers pour un projet WPF

- MonProjetWPF/

-  src/
 - -  MonProjet.UI (Projet WPF principal)
 -  Views/
 -  ViewModels/
 -  Models/
 -  Services/
 -  Resources/
 - -  Images/
 - -  Styles/
 - -  Templates/
 - -  MonProjet.Core (Bibliothèque de classes)
 -  Interfaces/
 -  Services/
 -  Models/
 - -  MonProjet.Tests (Tests unitaires)
 -  UnitTests/
 -  IntegrationTests/
-  docs/
 - -  README.md
 - -  Documentation technique